

Numerical Methods

Virgiliu Midrigan
New York University

Advanced Macroeconomics I

Motivation

- Consider standard incomplete markets model

$$V(a, e) = \max_{a'} \frac{c^{1-\theta}}{1-\theta} + \beta \int V(a', e') dF(e'|e) \quad \text{s.t.} \quad c + a' = (1+r)a + We$$

- Given guess of prices (r, W) , either directly maximize RHS
- Or solve Euler equation (policy function iteration)

$$c(a, e)^{-\theta} = \beta(1+r) \int c(a', e')^{-\theta} dF(e'|e)$$

- Then solve (r, W) to clear labor and asset markets

Motivation

- Numerical tasks involved:
 - solve non-linear equations, optimization
 - compute derivatives, integrals
 - function approximation; solving functional equations
- Today: introduce basic numerical tools to carry out these tasks
- Focus on efficiency and accuracy

Outline

① Basics

- linear and non-linear equations
- optimization
- differentiation and integration

② Functional approximation and solution to functional equations

- interpolation based on tensor product
- interpolation on sparse grids
- neural networks

Compecon Library

- <https://pfackler.wordpress.ncsu.edu/compecon/154-2/>
- Miranda and Fackler (2002)
- Implements most numerical tools I describe here
- Written in MATLAB with C-files for bottlenecks
 - make sure to compile .mex files for your specific OS
- Additional useful resources:
 - Judd (1998)
 - Press et. al. (1992) www.nr.com: Fortran and C routines

Linear Equations: $Ax = b$

- Many non-linear problems reduce to linear equations
- $\mathbf{x} = \mathbf{A} \setminus \mathbf{b}$ in MATLAB
 - very versatile, also least square solver if $\mathbf{A}$ not square
 - typically much more efficient than $\text{inv}(\mathbf{A}) * \mathbf{b}$, unless reuse many times
- Main problems
 - i) memory, though can exploit sparsity; ii) ill-conditioning (lack of precision)
 - condition number $\kappa(A) = \|A\| \times \|A^{-1}\|$ ($\text{cond}(\mathbf{A})$)
 - $\frac{\|x - x_{\text{true}}\|}{\|x_{\text{true}}\|} \leq \kappa(A) \cdot \epsilon_{\text{machine}}$, so $\kappa(A) \approx$ number of significant digits lost

Non-linear equations: $f(x) = 0$

Bisection (**bisect** in Compecon)

- 1-dimensional problems. Need continuity
- Choose bounds $[a, b]$ s.t. $\text{sgn} f(a) \neq \text{sgn} f(b)$. Let $c = \frac{a+b}{2}$
 - set $a' = a$ and $b' = c$ if $\text{sgn} f(b) = \text{sgn} f(c)$
 - set $a' = c$ and $b' = b$ if $\text{sgn} f(b) \neq \text{sgn} f(c)$
 - repeat until $b' - a' < \epsilon$
- Converges in $\log(\frac{b-a}{\epsilon})$ iterations. E.g., 10 to 10^{-7} in 27 iterations
- Also related Brent's method: faster due to quadratic interpolation

Non-linear equations: $f(x) = 0$

- Newton's method (**newton** in Compecon)

$$x^{i+1} = x^i - \left[\frac{\partial f(x^i)}{\partial x} \right]^{-1} f(x^i)$$

- need Jacobian info
 - very fast if it works, but can oscillate/diverge if f' changes sign too often
 - reduce step size if $\|f(x)\|$ increases after update
- Broyden's method (**broyden** in Compecon)

$$x^{i+1} = x^i - (A^i)^{-1} f(x^i)$$

$$A^{i+1} = A^i + [f(x^{i+1}) - f(x^i) - A^i(x^{i+1} - x^i)] \frac{(x^{i+1} - x^i)'}{(x^{i+1} - x^i)'(x^{i+1} - x^i)}$$

Function Iteration

- Find fixed point of a function $g(x)$ from R^n to R^n

$$x = g(x)$$

- Iteratively update $x' = g(x)$
- Converges if initial guess close to x^* and $\|g'(x^*)\| < 1$
- Can recast a root-finding problem as a fixed point problem

$$x = g(x) \equiv x - f(x)$$

Convergence rates

- Let $x^i \in R^n$ with $\lim_{i \rightarrow \infty} x^i = \bar{x}$
- x^i converges at rate q if $\lim_{i \rightarrow \infty} \frac{\|x^{i+1} - \bar{x}\|}{\|x^i - \bar{x}\|^q} < \infty$
 - e.g., $q = 2$ (quadratic): $\|x^i - \bar{x}\| = \frac{1}{2}, \frac{1}{4}, \frac{1}{16}, \frac{1}{256}, \dots, \frac{1}{2^{2^k}}$
- x^i converges linearly at rate β if $\lim_{i \rightarrow \infty} \frac{\|x^{i+1} - \bar{x}\|}{\|x^i - \bar{x}\|} < \beta < 1$
 - e.g., $\beta = \frac{1}{2}$: $\|x^i - \bar{x}\| = \frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{16}, \dots, \frac{1}{2^i}$
- Bisection: linear with $\beta = \frac{1}{2}$
- Function iteration: linear with $\beta = \left\| \frac{\partial f(\bar{x})}{\partial x} \right\|$
- Newton: quadratic
- Broyden: superlinear with $q \in (1, 2)$

Optimization: $\max_x f(x)$ where $f : R^n \rightarrow R$

- Golden search (**golden** in Compecon)
 - searches for maxima on $[a, b]$ in 1-dimension
 - given $x_1 < x_2$, replace $b = x_2$ if $f(x_1) > f(x_2)$ or $a = x_1$ if $f(x_2) > f(x_1)$
 - slow but sure
- Nelder-Mead (simplex, derivative-free): (**neldmead** in Compecon)
 - evaluate $f : R^n \rightarrow R$ at $n + 1$ points (simplex in n dimensions.)
 - reflect simplex away from point with lowest function value
 - slow and unreliable, but usually only tool available with many dimensions

Optimization: $\max_x f(x)$ where $f : R^n \rightarrow R$

- Quasi-Newton (**qnewton** in Compecon)

- Variants of Newton update:

$$x^{i+1} = x^i - \left[\frac{\partial^2 f(x)}{\partial x \partial x'} \right]^{-1} \frac{\partial f(x)}{\partial x}$$

- Hessian may be expensive, not negative-definite everywhere
- Replace with negative-definite approximations:
 - ① identity matrix (steepest ascent)
 - ② Davidson-Fletcher-Powell
 - ③ Broyden-Fletcher-Goldfarb-Shano

Differentiation

- Numerical derivatives: `fdjac` and `fdhess` in Compecon

$$f(x+h) = f(x) + f'(x)h + \frac{1}{2}f''(x)h^2 + O(h^3)$$

$$f(x-h) = f(x) - f'(x)h + \frac{1}{2}f''(x)h^2 + O(h^3)$$

$$f'(x) = \frac{f(x+h)-f(x-h)}{2h} + O(h^2)$$

- $h = 10^{-4}$ or 10^{-5} : as small as possible subject to numerical inaccuracies
- Automatic differentiation
 - break a function into multiple operations and apply chain rule
 - widely used in machine learning (e.g. tensorflow, also in matlab)

Integration

- We will mostly use Gaussian quadrature
 - other methods Newton-Cotes (simpson and trapezoid) also in Compecon
 - based on local approximations of f with line or cubic
 - `help qnwsimp` and `help qnwtrap`
- Gaussian quadrature for a given weighting function $w(x)$. Approximate

$$\int f(x)w(x)dx$$

- Choose nodes $x_1, \dots, x_n$ and weights $\omega_1, \dots, \omega_n$ to match $2n - 1$ moments:

$$\int x^k w(x)dx = \sum_{i=1}^n \omega_i x_i^k \text{ for } k = 0, \dots, 2n - 1$$

Gaussian Quadrature

- Choose nodes $x_1, \dots, x_n$ and weights $\omega_1, \dots, \omega_n$ to match $2n - 1$ moments:

$$\int x^k w(x) dx = \sum_{i=1}^n \omega_i x_i^k \text{ for } k = 0, \dots, 2n - 1$$

- Interpretation: discretize $w(x)$ with ω_i and mass points x_i
- Difficult numerical problem
- Specialized routines/tables for standard weighting functions available
- Integral approximated with

$$\int f(x) w(x) dx \approx \sum_{i=1}^n \omega_i f(x_i)$$

Gaussian Integration

- Weighting functions available in Compecon
 - Gauss-Legendre (**qnwlege**): $w(x) = 1$
 - Gauss-Chebyshev (**qnwcheb**): $w(x) = (1 - x^2)^{-\frac{1}{2}}$
 - Gaussian pdf: (**qnwnorm**)
 - Uniform pdf: (**qnwunif**)
 - Beta pdf: (**qnwbeta**)
 - Gamma pdf: (**qnwgamma**)
- Extension to > 1 dimension straightforward, but curse of dimensionality

$$\int_{a_2}^{b_2} \int_{a_1}^{b_1} f(x_1, x_2) dx_1 dx_2 \approx \sum_{i=1}^{n_2} \sum_{i=1}^{n_1} w_{1i} w_{2i} f(x_{1i}, x_{2i})$$

Example

- Compute $\mathbb{E} x y^{-2}$ where (x, y) are jointly log-normal

```
n      = [7, 7];
```

```
mean    = [0.1, 0.2];
```

```
varcov  = [0.2, 0.1; 0.1, 0.3];
```

```
[x,w]   = qnwnorm(n, mean, varcov);
```

```
answer  = w'*(exp(x(:,1) - 2*x(:,2))));
```

Function approximation: $f : D \subset R^k \rightarrow R^m$

- Goal: approximate f with $\hat{f}$ to minimize

$$\|f - \hat{f}\|_{\infty} = \sup_{x \in D} \|f(x) - \hat{f}(x)\|$$

- If f continuous, D closed and bounded, $\exists$ polynomial p s.t.

$$\|f - p\|_{\infty} < \epsilon$$

- Weierstrass theorem

- Function Interpolation:

- let $\hat{f}(x; c)$ linear combination of low-order polynomials with coefficients c
- choose c to min $\|f - \hat{f}\|$ at finite number of nodes in D
- choice of polynomials and nodes important

Choice of polynomials

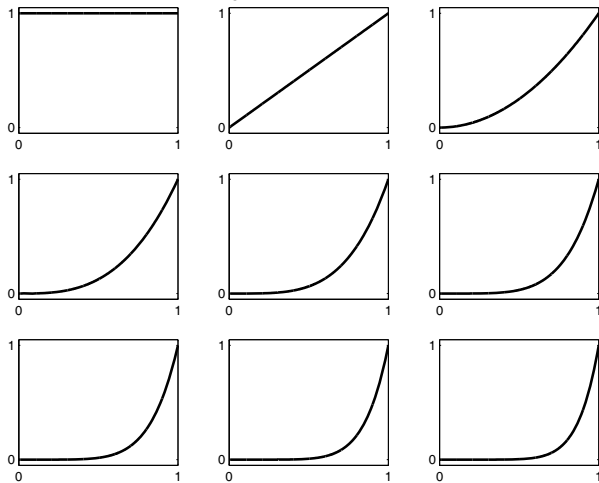
- Using $\hat{f}(x; c) = c_0 + c_1x + c_2x^2 + c_3x^3 + \dots$ bad idea
- Let $x = x_1, \dots, x_n$ be nodes for interpolation
- Choose c to ensure $\hat{f}(x; c) = f(x)$ (match values at n points)

$$\bullet \begin{pmatrix} 1 & x_1 & (x_1)^2 & \dots & (x_1)^n \\ 1 & x_2 & (x_2)^2 & \dots & (x_2)^n \\ 1 & x_3 & (x_3)^2 & \dots & (x_3)^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & (x_n)^2 & \dots & (x_n)^n \end{pmatrix} \begin{pmatrix} c_0 \\ c_1 \\ c_2 \\ \vdots \\ c_n \end{pmatrix} = \begin{pmatrix} f(x_1) \\ f(x_2) \\ f(x_3) \\ \vdots \\ f(x_n) \end{pmatrix}$$

- Matrix badly ill-conditioned (multi-collinear) for large n

Monomial basis functions

Figure 2: Monomial Basis Functions



Choice of polynomials

- Use orthogonal polynomials as basis functions instead
- Popular choice: Chebyshev polynomials

$$\phi_1(x) = 1$$

$$\phi_2(x) = x$$

$$\phi_3(x) = 2x^2 - 1$$

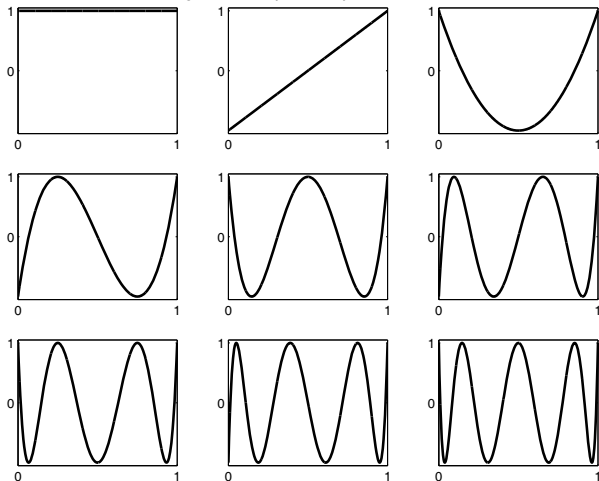
$$\phi_4(x) = 4x^3 - 3x$$

$$\phi_5(x) = 8x^4 - 8x^2 + 1$$

$$\phi_n(x) = 2x\phi_{n-1} - \phi_{n-2}$$

Chebyshev Basis Functions

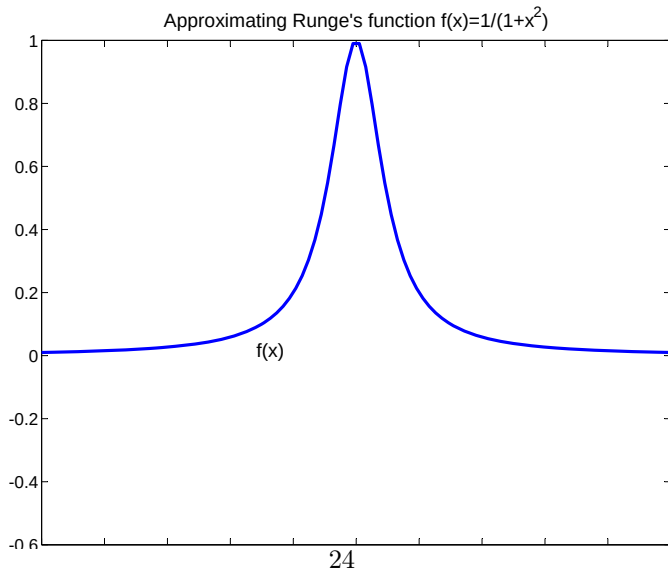
Figure 3: Chebyshev Polynomial Basis Functions



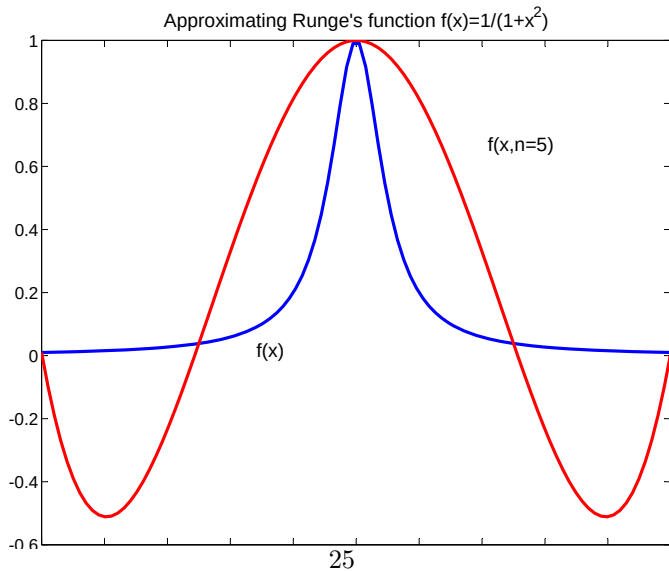
Choice of nodes

- Equidistant nodes bad idea
- Example: consider approximating Runge's function: $f(x) = \frac{1}{1+x^2}$

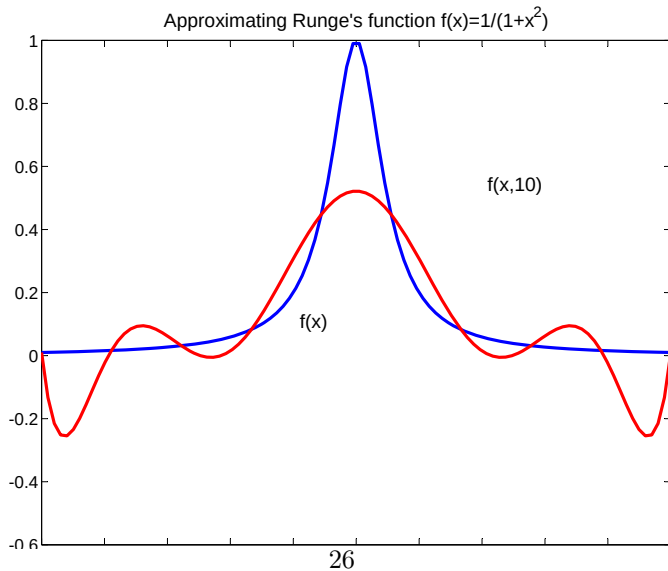
Equidistant nodes



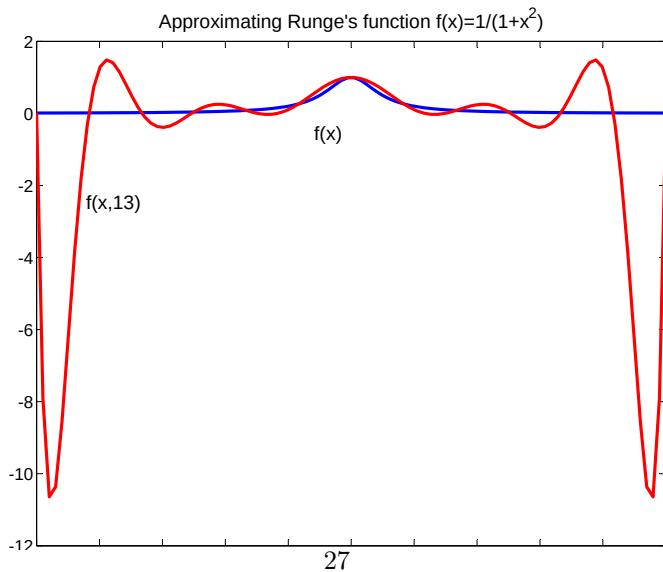
Equidistant nodes



Equidistant nodes



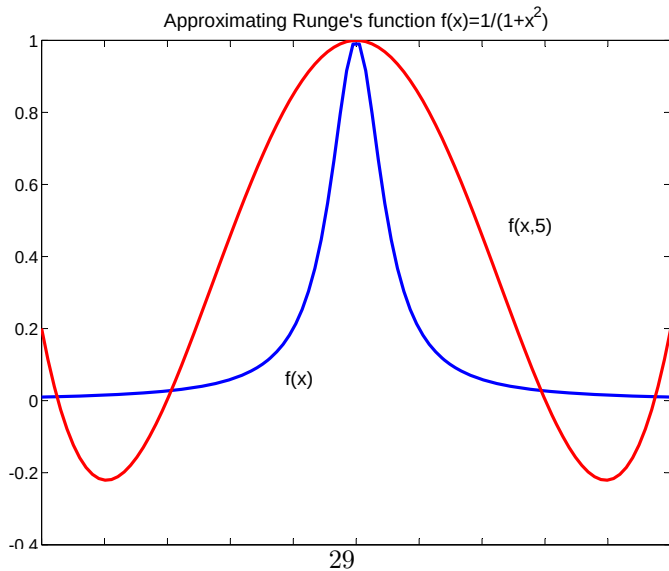
Equidistant nodes



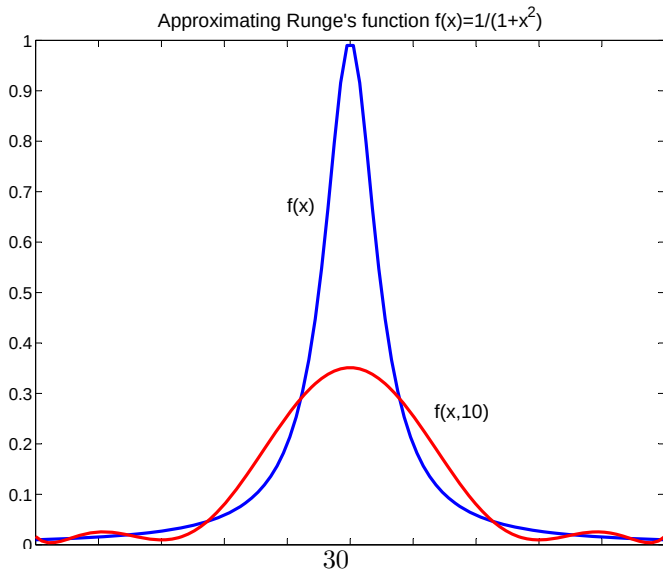
Optimal choice of interpolation nodes

- Chebyshev nodes (roots of n -th degree Chebyshev polynomial) optimal
- Rivlin's theorem:
 - If $f : [a, b] \rightarrow R$ is C^k for some $k \geq 1$ and I_n is interpolant at zeros of Chebyshev polynomials $\phi_n(x)$, then:
 - $\|f - I_n\|_\infty \leq \left(\frac{2}{\pi} \log(n+1) + 1\right) \frac{(n-k)!}{n!} \left(\frac{\pi}{2}\right)^k \left(\frac{b-a}{2}\right)^k \|f^{(k)}\|$
 - E.g., $k = 1$, bound = $\frac{(\frac{2}{\pi} \log(n+1) + 1)}{n} \frac{\pi}{4} \|f^{(1)}\| (b-a)$
- In words, approximation accurate if function sufficiently smooth

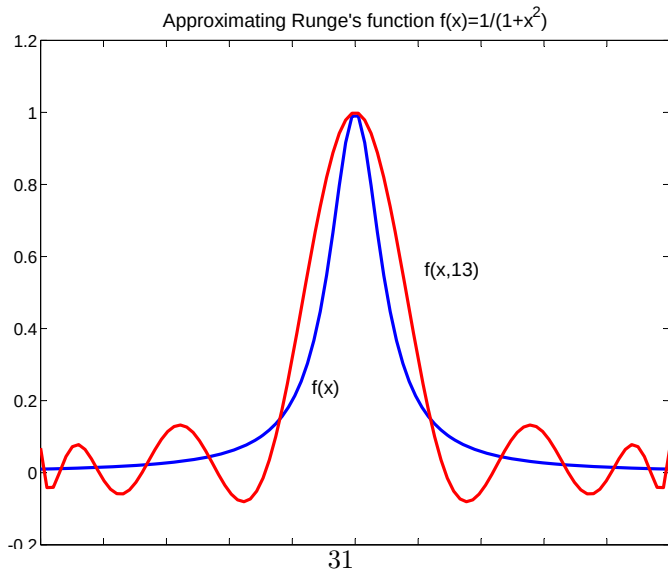
Chebyshev nodes



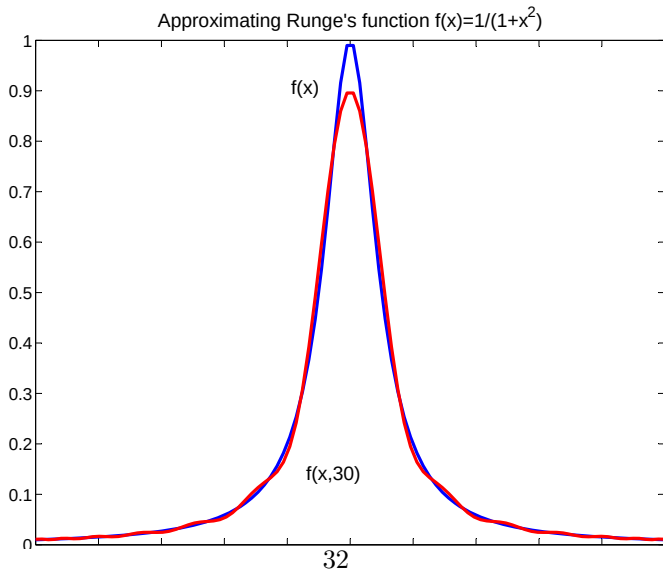
Chebyshev nodes



Chebyshev nodes



Chebyshev nodes



Solving for unknown coefficients

- Choose n : order of polynomials
- Choose $m \geq n$: number of nodes, $x = x_1, \dots, x_n$,
- Evaluate $F = f(x)$
- Choose $c = c_1, \dots, c_n$ to min: $(\Phi c - F)'(\Phi c - F)$

$$\Phi = \begin{pmatrix} \phi_1(x_1) & \phi_2(x_1) & \phi_3(x_1) & \dots & \phi_n(x_1) \\ \phi_1(x_2) & \phi_2(x_2) & \phi_3(x_2) & \dots & \phi_n(x_2) \\ \phi_1(x_3) & \phi_2(x_3) & \phi_3(x_3) & \dots & \phi_n(x_3) \\ \dots & \dots & \dots & \dots & \dots \\ \phi_1(x_m) & \phi_2(x_m) & \phi_3(x_m) & \dots & \phi_n(x_m) \end{pmatrix}$$

- Φ very well-conditioned: (use $\mathbf{c} = \mathbf{\Phi} \backslash \mathbf{F}$ in Matlab)

Example: Interpolate Runge's function on $[-10, 10]$

```
n          = 7;                                % order of polynomial
a          = -10; b = 10;                       % range of approximation

fspace     = fundef({'cheb', n, a, b});          % define function space: Chebyshev
x          = funnode(fspace);                   % generate Chebyshev nodes
Phi        = funbas(fspace, x);                 % evaluate basis functions at nodes

F          = 1./(1+s.^2);                       % evaluate function
c          = Phi\F;                             % coefficients

xx         = (a : .001 : b)';                  % dense grid
Phixx      = funbas(fspace, xx);                % basis at xx

plot(xx, [Phixx*c, 1./(1 + xx.^2)]);
```

Splines

- Order k *spline*: a series of k -th order polynomials spliced together
 - continuous first $k - 1$ derivatives
- Characterized by p *breakpoints*, $\nu_1, \dots, \nu_p$
 - points at which polynomials are spliced together
- $(p - 1)$ polynomials, so $(p - 1)(k + 1)$ coefficients
 - $(p - 2)(k - 1)$ pinned down by $(k - 1)$ smooth-pasting conditions
 - remaining $p + k - 1$ to match function values

Basis Functions

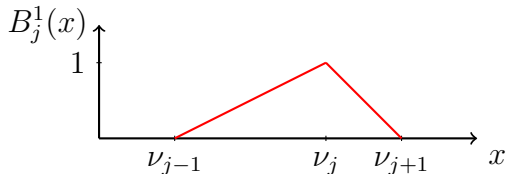
- Choose breakpoints $\nu_1, \dots, \nu_p$
 - put more in region where function has more curvature
- Construct $p + k - 1$ basis functions recursively using:
 - initialize $B_j^0(x) = 1$ if $\nu_j \leq x < \nu_{j+1}$, 0 otherwise (intercept in segment j)

$$B_j^k(x) = \frac{x - \nu_{j-k}}{\nu_j - \nu_{j-k}} B_{j-1}^{k-1}(x) + \frac{\nu_{j+1} - x}{\nu_{j+1} - \nu_{j-k+1}} B_j^{k-1}(x)$$

- automatically satisfies smooth-pasting conditions

Linear Splines ($k = 1$)

$$B_j^1(x) = \frac{x - \nu_{j-1}}{\nu_j - \nu_{j-1}} \mathbb{I}_{\{\nu_{j-1} \leq x < \nu_j\}} + \frac{\nu_{j+1} - x}{\nu_{j+1} - \nu_j} \mathbb{I}_{\{\nu_j \leq x < \nu_{j+1}\}}$$

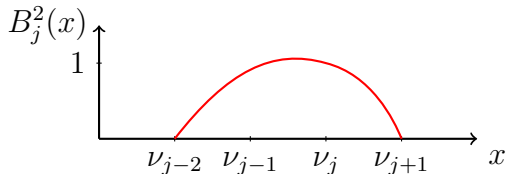


- Evaluated at the knots, $\Phi_{ij} = B_i^1(\nu_j) = \mathbb{I}\{i = j\}$, so $c_j = f(\nu_j)$

Example

- Breakpoints: 0, 2, 10
- $B(0.5) = [0.75, 0.25, 0]$ and $B(8) = [0, 0.25, 0.75]$
- Since $c = [f(0), f(2), f(10)]$
 - $\hat{f}(0.5) = 0.75 \times f(0) + 0.25 \times f(2)$
 - $\hat{f}(8.0) = 0.25 \times f(2) + 0.75 \times f(10)$

Quadratic Splines ($k = 2$)

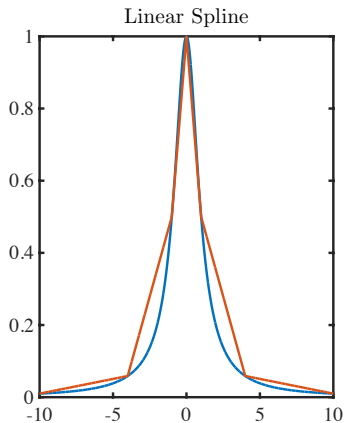
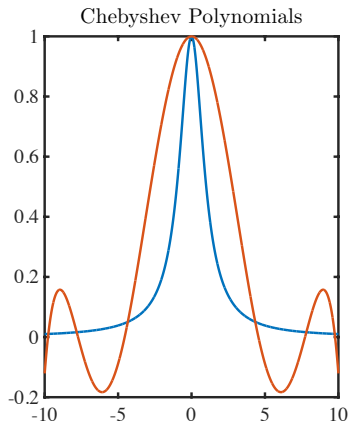


- Smooth curve (no kink at knots): higher-order splines increase smoothness
- Local support: $B_j^2(x) \neq 0$ only on $k + 2 = 4$ consecutive knots
- Sparsity: for any x , at most $k + 1 = 3$ quadratic basis functions are nonzero
- $\Phi_{ij} = B_j^2(x_i)$ is sparse $\Rightarrow$ fast projection and evaluation

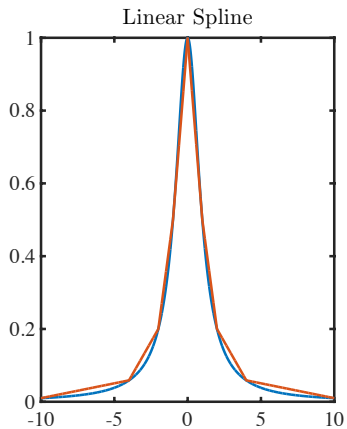
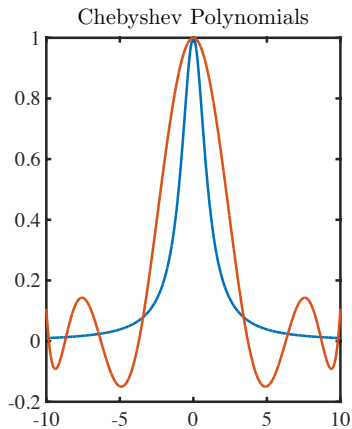
Advantages of Splines

- Interpolation:
 - non-linearity in some region does not lead to oscillations everywhere
 - can put more breakpoints in region with lots of curvature

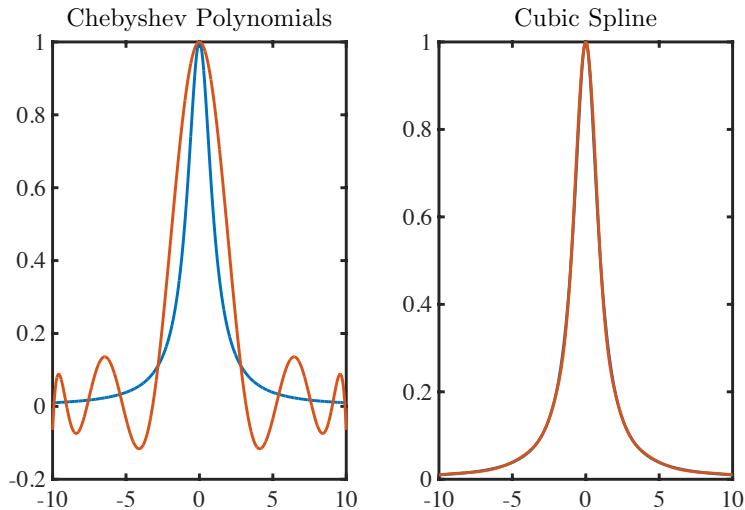
Chebyshev vs. Linear Splines, $n = 7$



Chebyshev vs. Linear Splines, $n = 9$



Chebyshev vs. Cubic Splines, $n = 11$



Example: Interpolate Runge's function using splines

```
a      = -10;
b      = 10;                                % range of approximation
splix  = 3;                                % order of spline (cubic)
vj     = [-10; -4; -2; -1; 0; 1; 2; 4; 10]; % breakpoints

fspace = fundef({'spli', vj, 0, splix});    % function space: cubic spline
x      = funnode(fspace);                  % nodes
Phi    = funbas(fspace, x);                % basis at nodes

F      = 1./(1 + x.^2);                    % Runge function
c      = Phi\F;                            % coefficients

xx     = (a : .001 : b)';                  % dense grid
Phixx  = funbas(fspace, xx);                % basis at xx

plot(xx, [Phixx*c, 1./(1 + xx.^2)]);
```

Extension to more dimensions

- Straightforward with tensor products, but curse of dimensionality
- E.g., approximate $f(x, y)$ on $[a_x, b_x] \times [a_y, b_y]$
- Generate univariate nodes $x = x_1, \dots, x_{n_x}; y = y_1, \dots, y_{n_y}$
- Compute univariate basis functions ϕ_i^x, ϕ_i^y
- Approximant:

$$\hat{f}(x, y) = \sum_{i_x=1}^{n_x} \sum_{i_y=1}^{n_y} c_{i_x i_y} \phi_{i_x}^x(x) \phi_{i_y}^y(y) = \left(\Phi_y(y) \otimes \Phi_x(x) \right) \times c$$

- More generally, in d dimensions: $\Phi = \Phi_d \otimes \Phi_{d-1} \otimes \dots \otimes \Phi_1$

Example in Compecon: $f(x, y) = \frac{1}{1+x^2y^2}$

```
n      = [7, 7];           % nodes in each dimension

a      = [-10, -10];
b      = [10, 10];         % bounds for each dimension

fspace = fundefn('cheb', n, a, b); % define function-approx space

xnode  = funnode(fspace);    % default nodes
x       = gridmake(xnode);   % Cartesian product of nodes

Phi     = funbas(fspace, x, [0, 0]); % basis at grid (derivative order)

F       = 1./(1 + x(:, 1).^2 .* x(:, 2).^2); % f at nodes
c       = Phi\F;             % coefficients
```

Multiple Dimensions

- Similar syntax for splines:

```
fspace = fundef({'spli', xn, 0, 1}, {'spli', yn, 0, 3});  
s      = gridmake(funnode(fspace));  
Phi    = funbas(fspace, s);
```

- Can mix splines and Chebyshev:

```
fspace = fundef({'cheb', 7, -10, 10}, {'spli', yn, 0, 3});  
s      = gridmake(funnode(fspace));  
Phi    = funbas(fspace, s);
```

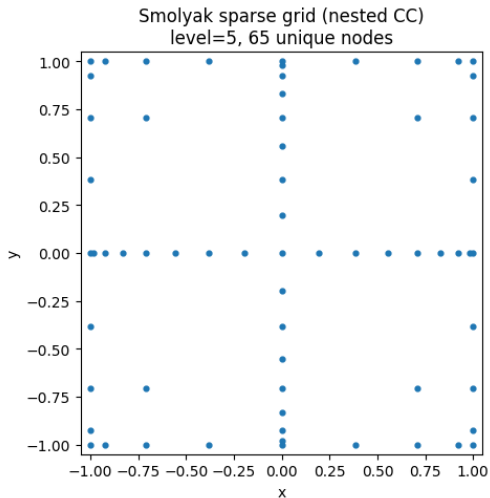
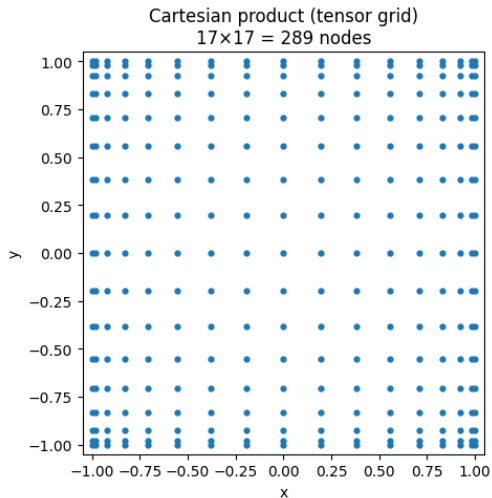
Curse of dimensionality

- Number of nodes/coefficients grows exponentially with dimension
- E.g. from my paper with Corina and Denis on housing
 - liquid assets: 75 nodes
 - mortgage debt: 55 nodes
 - house size: 7 nodes
 - persistent income : 7 nodes
 - transitory income: 3 nodes
 - age: 240 quarters
 - total: $75 \times 55 \times 7 \times 7 \times 3 \times 240 = 145,530,000$ nodes/coefficients

Smolyak sparse grids

- Discard high-order polynomials/nodes
 - e.g, keep $x^3, x^4, y^3, y^4, x^2y^2, x^3y, xy^3$ but discard $x^4y, xy^4, x^3y^2, x^2y^3$
- Suggested by Smolyak (1963), applied in economics by Kubler and Kruger
 - Judd, Maliar, Maliar, Valero (2014): Chebyshev polynomials
 - Klimke's Matlab Toolbox also implements linear interpolation
 - Brumm, Scheidegger: adaptive grid (add nodes region with larger errors)

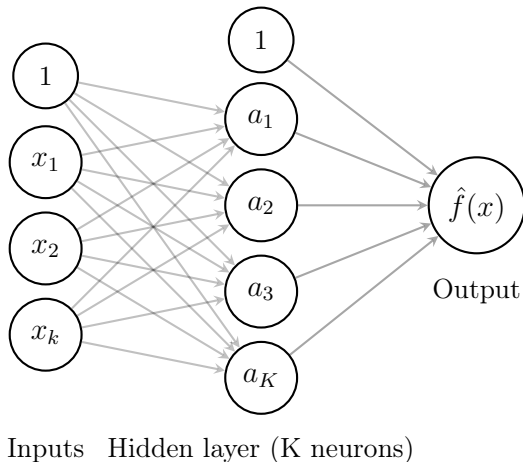
Example with $d = 2$



Neural Networks

- Gaining popularity with faster GPU computing
- Key idea: don't pick basis functions. Adapt basis to the shape of function
- Advantage: more flexible, mitigates curse of dimensionality
- Disadvantage: no longer linear projection. Need non-linear optimization

Neural Networks: Architecture (1 hidden layer)



- Weights on arrows are the coefficients we need to pin down

What a Neuron Computes

$$z = Wx + b$$

$$a = g(z)$$

linear index z then nonlinearity g

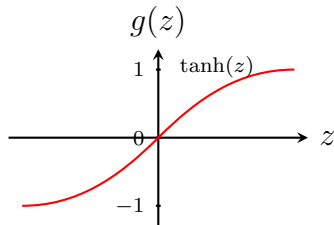
- Each hidden unit takes linear combination of inputs, then applies an activation
- With K neurons: $a(x) = g^1(W^1x + b^1)$ where $a(x) \in \mathbb{R}^K$
- Output layer combines these features: $\hat{f}(x) = g^2(W^2a(x) + b^2)$

Activation Functions

- If g linear, network collapses to line

- Common choices:

- ReLU: $g(z) = \max\{0, z\}$
- Tanh: $g(z) = \tanh(z)$
- Softplus: $g(z) = \log(1 + e^z)$
- Logistic: $g(z) = \frac{1}{1+e^{-z}}$



Simple Example

- Return to Runge's function $f(x) = \frac{1}{1+x^2}$
- Try baby network with one hidden layer with 2 neurons:

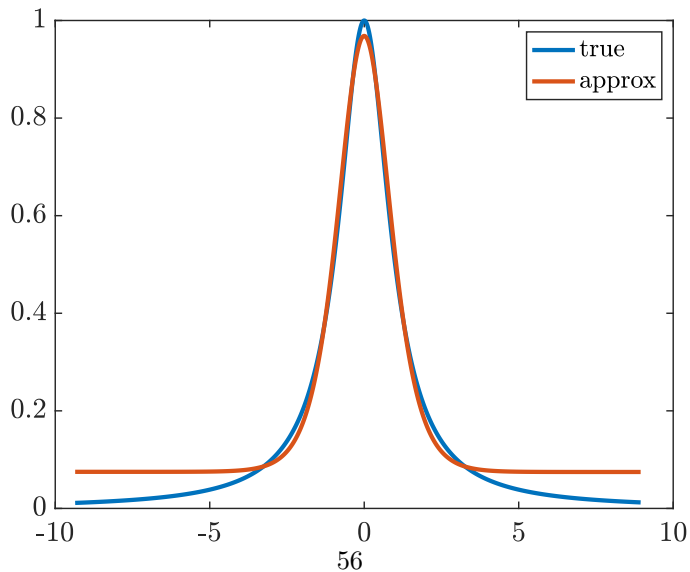
$$\hat{f}(x) = \gamma_0 + \gamma_1 a_1(x) + \gamma_2 a_2(x)$$

$$a_1(x) = \frac{1}{1 + \exp(b_1 + w_1 x)}$$

$$a_2(x) = \frac{1}{1 + \exp(b_2 + w_2 x)}$$

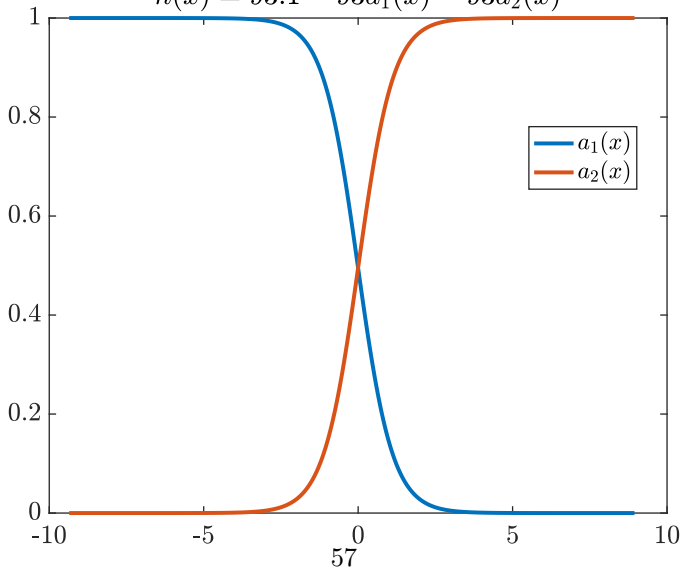
- Train (minimize mse) using 1000 randomly drawn x

Results with 2 neurons

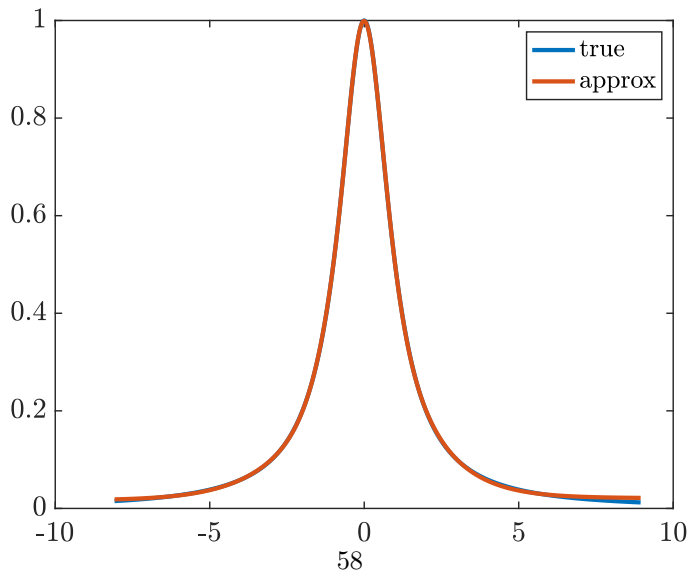


Basis functions $a_1(x)$ and $a_2(x)$

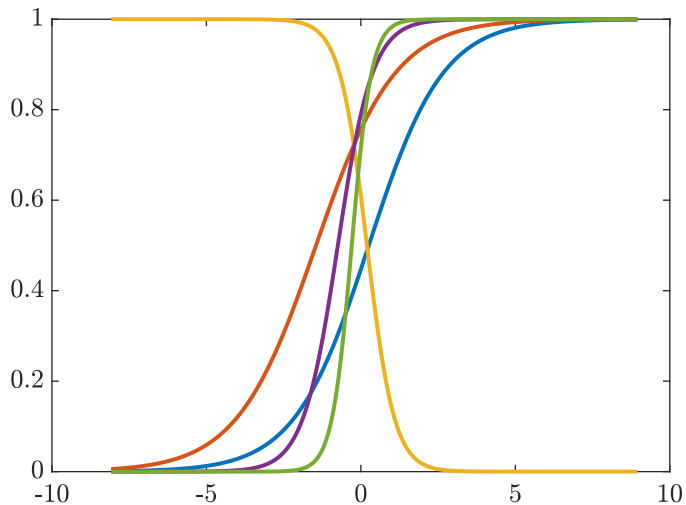
$$h(x) = 93.1 - 93a_1(x) - 93a_2(x)$$



Results with 5 neurons



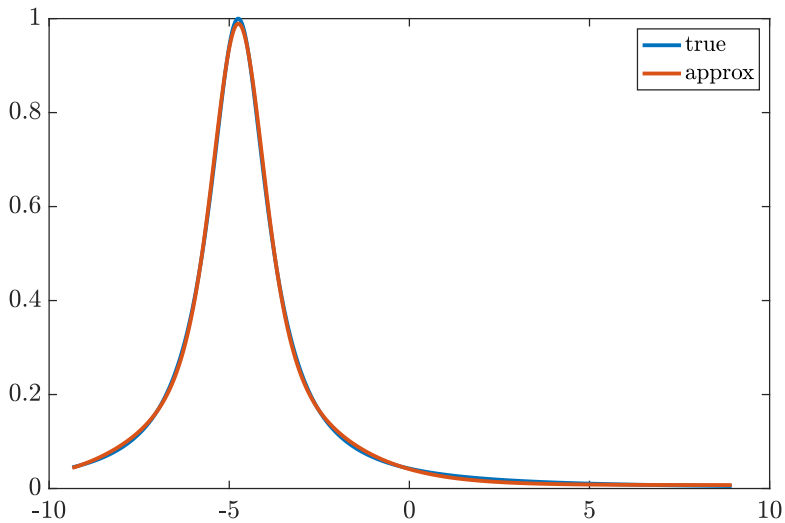
Basis functions with 5 neurons



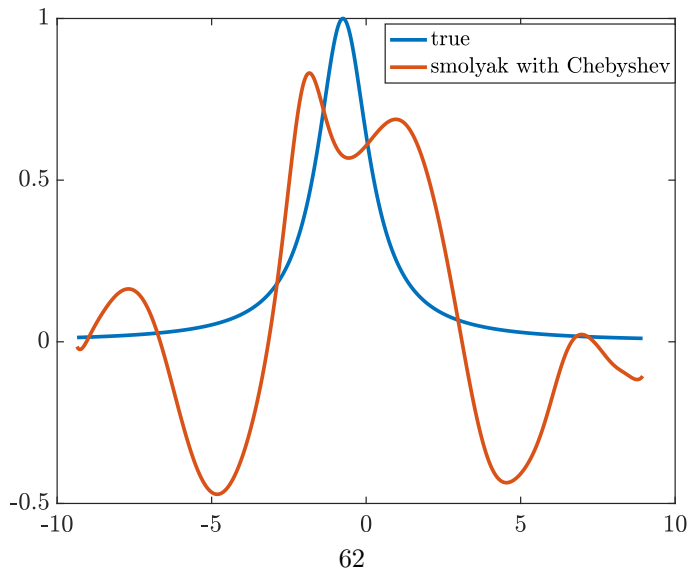
Curse of Dimensionality

- Suppose x has 20 elements. $f(x) = 1/(1 + \sum_{i=1}^{20} x_i^2)$
- With 5 hidden neurons, θ has $5 \times 21 + 6 = 111$ elements
- Tensor product with 3 polynomials in each dim has $3^{20} = 3$ billion
- Smolyak poor for functions with lots of curvature

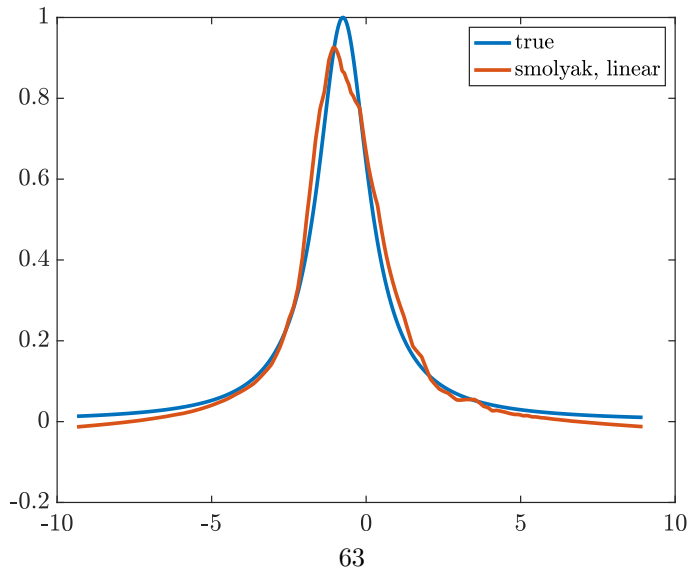
$$f(x_1, x_2, \dots, x_{20} = 0.25)$$



Smolyak with Chebyshev ($N = 4$)



Smolyak Piecewise Linear ($N = 4,470,000$ nodes)



Training the Model

- Choose coefficients θ to minimize

$$J(\theta) = \frac{1}{2N} \sum_{i=1}^N (h(x^i, \theta) - f(x^i))^2$$

- Gradient

$$\nabla_{\theta} J(\theta) = \frac{1}{N} \sum_{i=1}^N (h(x^i, \theta) - f(x^i)) \nabla_{\theta} h(x^i, \theta)$$

- $\nabla_{\theta} h(x^i, \theta)$ available in closed form using back-propagation (chain rule)
 - use automatic differentiation for more complicated problems
- Gradient descent: $\theta_{t+1} = \theta_t - \alpha_t \nabla_{\theta} J(\theta_t)$

Stochastic Gradient Descent

- Machine learning applications usually require lots of data (large N)
- Computing gradient for entire data may be very expensive
- Stochastic gradient descent
 - choose a small batch of, say, 512 points
 - update θ evaluating gradient at only those points
- Various algorithms, Adam most popular
 - moving average of gradients (momentum)

Solving Functional Equations

- Straightforward using tools above
- Basic idea: suppose we want to solve for $f(x)$ such that $T(f) = 0$
- Approximate $f(x)$ with $\Phi(x)c$ where $\Phi(x)$ are basis functions
- Solve for c : $T(\Phi(x)c) = 0$ at e.g. Chebyshev nodes in state-space
 - turns a functional equation into a system of n nonlinear equations

Simple Example

- Consider a consumer who cannot trade so consumes income:

$$V(y) = u(y) + \beta \int V(\rho y + \varepsilon) dF(\varepsilon)$$

- Approximate $V(y) = \Phi(y)c$ and use Gaussian quadrature

$$\Phi(y)c = u(y) + \beta \sum_{i=1}^K w_i \Phi(\rho y + \varepsilon_i) c$$

- Let $\sum_{i=1}^K w_i \Phi(\rho y + \varepsilon_i) = \bar{\Phi}(y)$

$$c = [\Phi(y) - \beta \bar{\Phi}(y)]^{-1} u(y)$$

Solving HJB equation

- Suppose want to solve

$$rv(x) = \pi(x) + \mu(x)v'(x) + \frac{\sigma^2(x)}{2}v''(x)$$

- With boundary conditions

$$v(s) = v(S) = v(x^*) - \xi$$

- Linear system

$$\left[r\Phi(x) - \mu(x)\Phi'(x) - \frac{\sigma^2(x)}{2}\Phi''(x) \right] c = \pi(x)$$

$$[\Phi(s) - \Phi(x^*)] c = -\xi$$

$$[\Phi(S) - \Phi(x^*)] c = -\xi$$

Example with a Savings Choice

- Suppose can save with risk-free asset:

$$V(a, y) = \max_{a'} u((1+r)a + y - a') + \beta \int V(a', \rho y + \varepsilon) dF(\varepsilon)$$

- Solve iteratively. Given guess for c , solve for a' using e.g. golden search

$$\max_{a'} u((1+r)a + y - a') + \beta \sum_{i=1}^K w_i \Phi(a', \rho y + \varepsilon_i) c$$

- Let $\sum_{i=1}^K w_i \Phi(a'^*(a, y), \rho y + \varepsilon_i) = \bar{\Phi}(a, y)$
- Update c using Howard Policy Improvement and use in next iteration

$$c = [\Phi(y) - \beta \bar{\Phi}(a, y)]^{-1} u(y)$$

Neural Networks

- Similar to how we fit a function, now minimize errors in equil conditions
- E.g. choose θ to minimize

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N \left(c(a_i, z_i; \theta)^{-\sigma} - \beta(1+r) \mathbb{E} c(a'_i, z'_i; \theta)^{-\sigma} \right)^2$$

$$a' = (1+r)a_i + \exp(z_i) - c(a_i, z_i; \theta)$$

- Advantage: can choose points (a_i, z_i) visited often, not Cartesian grid